CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

Milestone #: 2

Date: March 3, 2025

Group Number: Group 30

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Cillian Dong	66679796	y4u4k	dongbaiqi22@gmail.com
Jason JI	69217222	z0v1e	sji05@student.ubc.ca
Hongkai Liu	70744453	**n2o4u**	liuhongkai112233@163.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

---

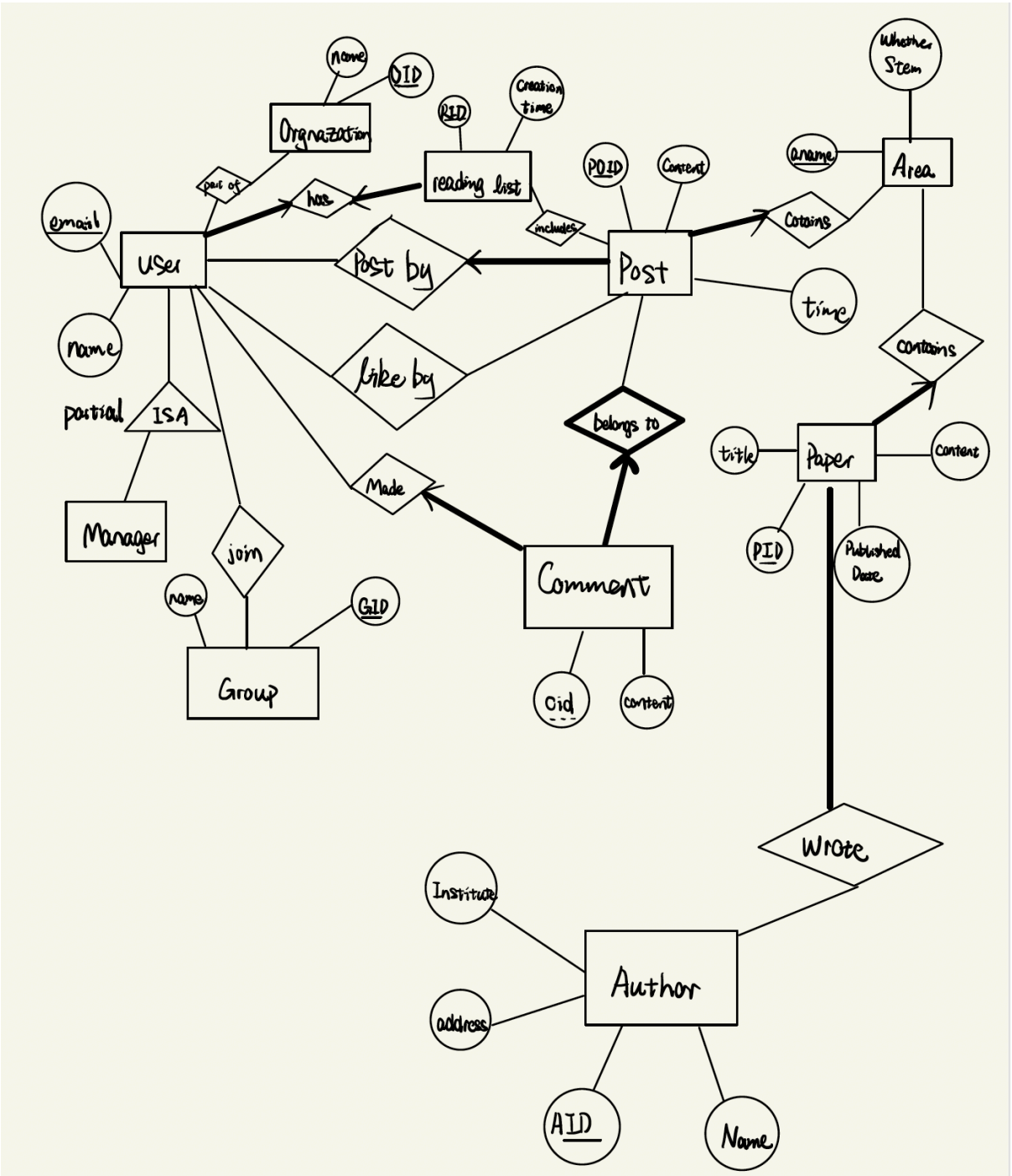
2. Brief Project Summary

Our project is an academic discussion platform designed to facilitate communication among researchers and scholars. It enables users to share insights, discuss research topics, and connect with peers in their academic fields. The platform supports structured discussions categorized by disciplines, interest-based groups, and bookmarking features, helping users efficiently engage in meaningful research discussions.

3. ER Diagram(changed)

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science



Note for Changes Made to ER Diagram:

1. Added two new attributes content and title to entity Paper. It makes more sense to also include the content and the title of the paper rather than solely its id and published time.
2. Added two new attributes institute and address to entity Author. Author belongs to some institute and we also want to record the address for each author.
3. Before, for each entity, the attributes refer to id all had the same name “id”. To clear this confusion, we added a prefix to each id.

4. Before, there were multiple relationships having the same name “belongs to”, which may lead to confusions in the future. Hence, we changed the name of those relationships that had the same name to avoid any potential confusions.
5. We changed the relationship between Post and Comment from ISA to Comment is a weak entity of Comment. This can preserve the relation that “post owns comments”.

4. Schemas:

User (email:VARCHAR, name:VARCHAR, **rid**:INTEGER): rid not null, rid unique, rid references
Manager(**email**:VARCHAR): email references User
ReadingList(rid:INTEGER, creationTime:DATE)
Organization(oid:INTEGER, name:VARCHAR)
PartOf(**email:VARCHAR,oid:VARCHAR**): email references User, oid references organization
Group(gid:INTEGER, name:VARCHAR)
Join(**gid:INTEGER, emai:VARCHAR**) email references User, gid references Group
Post(poid:INTEGER, content:VARCHAR, time:TIME, **email**:VARCHAR, **aname**:VARCHAR): email not null, aname not null, email references User, aname references Area
Includes(**rid:INTEGER, poid:INTEGER**)
Area(aname:VARCHAR, whetherStem:BOOLEAN)
Paper(pid:INTEGER, publishedDate:DATE, content:VARCHAR, **aname**:VARCHAR, title:VARCHAR) aname not null, aname references Area
Author(aid:INTEGER, name:VARCHAR, instituteName:VARCHAR, address:VARCHAR)
Wrote(**aid:INTEGER,pid:INTEGER**) pid references Paper, aid references Author
Comment(**poid:INTEGER**, cid:INTEGER, **email**:VARCHAR, content:VARCHAR): email not null, email references User, poid references Post
LikedBy(**poid:INTEGER, emai:VARCHARl**): email references User, poid references Post

5. Functional Dependency(FDs):

User: email is the PK
email -> name
email -> rid

Manager:
No FDs

ReadingList: eid is the PK
rid ->creationTime

Organization: oid is the PK

---

oid -> name

PartOf:
No FDs

Group: gid is the PK
gid -> name

Join:
No FDs

Post: poid is the PK
poid -> content
poid -> time
poid -> email
poid -> aname

Includes:
No FDs

Area: aname is the PK
aname -> whetherStem

Paper: pid is the PK
pid -> publishedDate
pid -> content
pid -> aname
pid -> title

Author: aid is the PK
aid -> name
instituteName -> address(violate BCNF)

Wrote:
No FDs

Comment: poid, cid is the PK
poid, cid -> email
poid, cid -> content

LikedBy:
No FDs

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

---

6. Normalization:

In BCNF, for any functional dependency X -> Y, X must be a superkey. All relations except the Author relation are already in BCNF, because all functional dependencies (FDs) in these relations have the left-hand side as a superkey, and their closure covers all attributes within their respective relations.

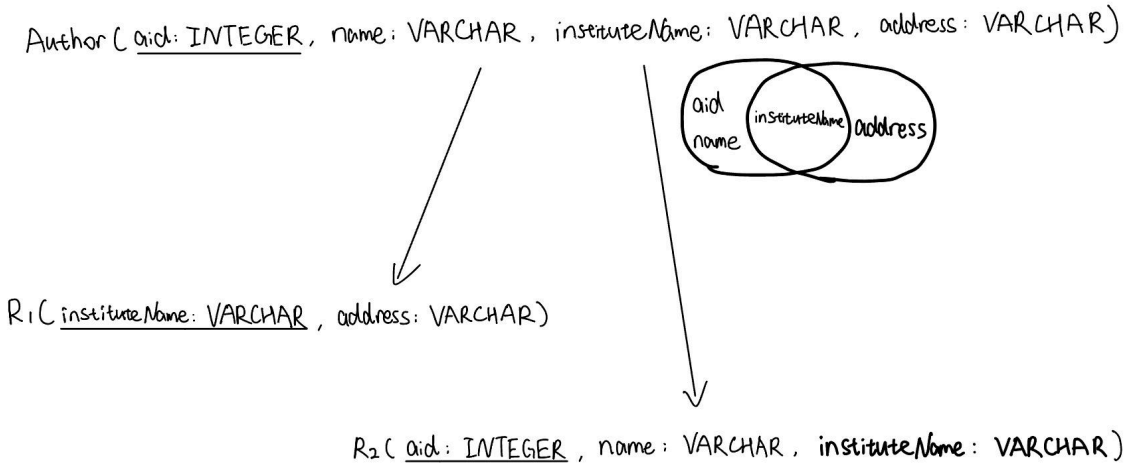
Given the following functional dependencies in the relation Author(aid, name, instituteName, address):

1. aid -> name
2. instituteName -> address(violate BCNF)

Since aid is the primary key, the first FD is already in BCNF because aid is already a superkey.

However, in the second FD, the closure of instituteName is {instituteName, address}, which means instituteName is not a superkey. Since BCNF requires that the left-hand side of all FDs be a superkey, this dependency violates BCNF.

Here is the process of the BCNF Decomposition:



In R1, since instituteName → address, instituteName is the primary key of R1, making it in BCNF.

In R2, since aid is the primary key in R2, and it determines all other attributes (aid → name, instituteName), R2 is also in BCNF.

So the final result is R1(instituteName:VARCHAR, address:VARCHAR), R2(aid:INTEGER, name:VARCHAR, **instituteName:VARCHAR**)

7. SQL DDL statements:

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

```
CREATE TABLE User (
   email VARCHAR PRIMARY KEY,
   name VARCHAR,
   rid INTEGER UNIQUE,
   FOREIGN KEY (rid)
      REFERENCES ReadingList(rid)
      ON DELETE SET NULL
      ON UPDATE CASCADE
);
```

Foreign Key Constraints:

FOREIGN KEY (rid) REFERENCES ReadingList(rid) ON DELETE CASCADE ON UPDATE CASCADE

ON DELETE SET NULL → If a ReadingList is deleted, rid in User is set to NULL, ensuring the user remains but loses the reference.
ON UPDATE CASCADE → If rid is updated in ReadingList, it will be updated in User.

```
CREATE TABLE Manager (
   email VARCHAR PRIMARY KEY,
   FOREIGN KEY (email)
      REFERENCES User(email)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

Foreign Key Constraints:

FOREIGN KEY (email) REFERENCES User(email) ON DELETE CASCADE ON UPDATE CASCADE

ON DELETE CASCADE → When a User is deleted, the corresponding Manager is also deleted.
ON UPDATE CASCADE → When a User's email is updated, the corresponding Manager's email is also updated.

```
CREATE TABLE ReadingList (
   rid INTEGER PRIMARY KEY,
   creationTime DATE
);
```

No any Foreign Key Constraints.

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

```
CREATE TABLE Organization (
   oid INTEGER PRIMARY KEY,
   name VARCHAR
);
```

No any Foreign Key Constraints.

```
CREATE TABLE PartOf (
   email VARCHAR,
   oid INTEGER,
   PRIMARY KEY (email, oid),
   FOREIGN KEY (email)
      REFERENCES User(email)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
   FOREIGN KEY (oid)
      REFERENCES Organization(oid)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

Foreign Key Constraints:

FOREIGN KEY (email) REFERENCES User(email) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a user is deleted from the User table, their associated records in PartOf are also deleted, ensuring no orphaned references.
ON UPDATE CASCADE → If a user's email is updated in the User table, the corresponding email in PartOf is automatically updated, maintaining consistency.

FOREIGN KEY (oid) REFERENCES Organization(oid) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If an organization is deleted from the Organization table, all PartOf records referencing that organization are also deleted, ensuring referential integrity.
ON UPDATE CASCADE → If an organization's oid is updated in the Organization table, the oid in PartOf is automatically updated accordingly.

```
CREATE TABLE Group (
   gid INTEGER PRIMARY KEY,
   name VARCHAR
);
```

CPSC 304 Project Cover Page

University of British Columbia, Vancouver

Department of Computer Science

No any Foreign Key Constraints.

```
CREATE TABLE Join (
   gid INTEGER,
   email VARCHAR,
   PRIMARY KEY (gid, email),
   FOREIGN KEY (gid)
      REFERENCES Group(gid)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
   FOREIGN KEY (email)
      REFERENCES User(email)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

Foreign Key Constraints

FOREIGN KEY (gid) REFERENCES Group(gid) ON DELETE CASCADE ON UPDATE CASCADE

ON DELETE CASCADE → If a group is deleted, all its membership records in Join will be deleted.

ON UPDATE CASCADE → If gid is updated in Group, it will be updated in Join.

FOREIGN KEY (email) REFERENCES User(email) ON DELETE CASCADE ON UPDATE CASCADE

ON DELETE CASCADE → If a user is deleted, their membership records will be deleted from Join.

ON UPDATE CASCADE → If email is updated in User, it will be updated in Join.

```
CREATE TABLE Post (
   rid INTEGER PRIMARY KEY,
   poid INTEGER,
   content VARCHAR,
   time TIME,
   email VARCHAR NOT NULL,
   aname VARCHAR NOT NULL,
   FOREIGN KEY (email)
      REFERENCES User(email)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
   FOREIGN KEY (aname)
      REFERENCES Area(aname)
```

```
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Foreign Key Constraints:

FOREIGN KEY (email) REFERENCES User(email) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a user is deleted from User, all posts they authored will be automatically deleted.
ON UPDATE CASCADE → If email is updated in User, it will be automatically updated in Post.

FOREIGN KEY (aname) REFERENCES Area(aname) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If an Area is deleted, all posts associated with that area will also be deleted.
ON UPDATE CASCADE → If aname is updated in Area, it will be automatically updated in Post.

```
CREATE TABLE Includes (
  rid INTEGER,
  poid INTEGER,
  PRIMARY KEY (rid, poid),
  FOREIGN KEY (rid)
    REFERENCES ReadingList(rid)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  FOREIGN KEY (poid)
    REFERENCES Post(poid)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Foreign Key Constraints:

FOREIGN KEY (rid) REFERENCES ReadingList(rid) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a reading list is deleted, all related Includes records will be deleted.
ON UPDATE CASCADE → If rid is updated in ReadingList, it will be updated in Includes.

FOREIGN KEY (poid) REFERENCES Post(poid) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a post is deleted, all records linking it to a reading list will be deleted.
ON UPDATE CASCADE → If poid is updated in Post, it will be updated in Includes.

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

```
CREATE TABLE Area (
   aname VARCHAR PRIMARY KEY,
   whetherStem BOOLEAN
);
```
No any Foreign Key Constraints.

```
CREATE TABLE Paper (
   pid INTEGER PRIMARY KEY,
   publishedDate DATE,
   content VARCHAR,
   aname VARCHAR,
   title VARCHAR,
   FOREIGN KEY (aname)
      REFERENCES Area(aname)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

Foreign Key Constraints:

FOREIGN KEY (aname) REFERENCES Area(aname) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If an area is deleted, all papers associated with it will be deleted.
ON UPDATE CASCADE → If aname is updated in Area, it will be updated in Paper.

```
CREATE TABLE Institute (
   instituteName VARCHAR PRIMARY KEY,
   address VARCHAR
);
```
No any Foreign Key Constraints.

```
CREATE TABLE Author (
   aid INTEGER PRIMARY KEY,
   name VARCHAR,
   instituteName VARCHAR NOT NULL,
   FOREIGN KEY (instituteName)
      REFERENCES Institute(instituteName)
      ON DELETE CASCADE
      ON UPDATE CASCADE
```

);
Foreign Key Constraints:

FOREIGN KEY (instituteName) REFERENCES Institute(instituteName) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If an institute is deleted, all its authors will be deleted.
ON UPDATE CASCADE → If instituteName is updated in Institute, it will be updated in Author.

```
CREATE TABLE Wrote (
   aid INTEGER,
   pid INTEGER,
   PRIMARY KEY (aid, pid),
   FOREIGN KEY (aid)
      REFERENCES Author(aid)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
   FOREIGN KEY (pid)
      REFERENCES Paper(pid)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

Foreign Key Constraints:

FOREIGN KEY (aid) REFERENCES Author(aid) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If an author is deleted, all Worte instances related to the author will be automatically deleted.
ON UPDATE CASCADE → If aid is updated in Author, it will be updated in Wrote.
FOREIGN KEY (pid) REFERENCES Paper(pid) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a paper is deleted, all their Worte instances will be automatically deleted.
ON UPDATE CASCADE → If pid is updated in Paper, it will be updated in Wrote.

```
CREATE TABLE Comment (
   poid INTEGER,
   cid INTEGER,
   email VARCHAR NOT NULL,
   content VARCHAR,
   PRIMARY KEY (poid, cid),
   FOREIGN KEY (poid)
      REFERENCES Post(poid)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
   FOREIGN KEY (email)
```

```
    REFERENCES User(email)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Foreign Key Constraints

FOREIGN KEY (poid) REFERENCES Post(poid) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a post is deleted, all its comments will be automatically deleted.
ON UPDATE CASCADE → If poid is updated in Post, it will be updated in Comment.
FOREIGN KEY (email) REFERENCES User(email) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a user is deleted, all their comments will be automatically deleted.
ON UPDATE CASCADE → If email is updated in User, it will be updated in Comment.

```
CREATE TABLE LikedBy (
  poid INTEGER,
  email VARCHAR,
  PRIMARY KEY (poid, email),
  FOREIGN KEY (poid)
    REFERENCES Post(poid)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  FOREIGN KEY (email)
    REFERENCES User(email)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Foreign Key Constraints

FOREIGN KEY (poid) REFERENCES Post(poid) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a post is deleted, all its likes will be automatically deleted.
ON UPDATE CASCADE → If poid is updated in Post, it will be updated in LikedBy.
FOREIGN KEY (email) REFERENCES User(email) ON DELETE CASCADE ON UPDATE CASCADE
ON DELETE CASCADE → If a user is deleted, all their likes will be automatically deleted.
ON UPDATE CASCADE → If email is updated in User, it will be updated in LikedBy.

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

---

8. Insertion Statements:

Table User:

```
INSERT INTO User (email, name, rid) VALUES
('alice@example.com', 'Alice', 1),
('bob@example.com', 'Bob', 2),
('charlie@example.com', 'Charlie', 3),
('david@example.com', 'David', 4),
('eve@example.com', 'Eve', 5),
('frank@example.com', 'Frank', 6),
('grace@example.com', 'Grace', 7),
('henry@example.com', 'Henry', 8),
('ivy@example.com', 'Ivy', 9),
('jack@example.com', 'Jack', 10);
```

Table Manager:

```
INSERT INTO Manager (email) VALUES
('alice@example.com'),
('bob@example.com'),
('charlie@example.com'),
('grace@example.com'),
('henry@example.com');
```

Table ReadingList:

```
INSERT INTO ReadingList (rid, creationTime) VALUES
(1, '2024-03-01'),
(2, '2024-03-02'),
(3, '2024-03-03'),
(4, '2024-03-04'),
(5, '2024-03-05');
```

Table Organization:

```
INSERT INTO Organization (oid, name) VALUES
(1, 'Tech Innovators Inc.'),
(2, 'Global Finance Ltd.'),
(3, 'Health & Wellness Co.'),
(4, 'Green Energy Solutions'),
(5, 'AI Research Lab');
```

Table PartOf:

```
INSERT INTO PartOf (email, oid) VALUES
('alice@example.com', 1),
('bob@example.com', 2),
('charlie@example.com', 3),
```

('david@example.com', 4),
('eve@example.com', 5);

Table Group:
INSERT INTO Group (gid, name) VALUES
(1, 'Data Science Enthusiasts'),
(2, 'Software Developers Hub'),
(3, 'Cybersecurity Experts'),
(4, 'AI & Machine Learning Society'),
(5, 'Cloud Computing Innovators');

Table Join:
INSERT INTO Join (gid, email) VALUES
(1, 'alice@example.com'),
(2, 'bob@example.com'),
(3, 'charlie@example.com'),
(4, 'david@example.com'),
(5, 'eve@example.com');

Table Area:
INSERT INTO Area (aname, whetherStem) VALUES
('Tech', TRUE),
('Finance', TRUE),
('Health', TRUE),
('AI', TRUE),
('Energy', TRUE);

Table Post:
INSERT INTO Post (rid, poid, content, time, email, aname) VALUES
(1, 101, 'First post in the tech area', '10:30:00', 'alice@example.com', 'Tech'),
(2, 102, 'Financial market trends today', '11:15:00', 'bob@example.com', 'Finance'),
(3, 103, 'Health benefits of regular exercise', '12:45:00', 'charlie@example.com', 'Health'),
(4, 104, 'Latest developments in AI research', '14:00:00', 'david@example.com', 'AI'),
(5, 105, 'Sustainable energy innovations', '15:20:00', 'eve@example.com', 'Energy');

Table Includes:
INSERT INTO Includes (rid, poid) VALUES
(1, 101),
(2, 102),
(3, 103),
(4, 104),
(5, 105);

Table Paper:

CPSC 304 Project Cover Page

University of British Columbia, Vancouver

Department of Computer Science

INSERT INTO Paper (pid, publishedDate, content, aname, title) VALUES
(1, '2024-01-10', 'Exploring AI Ethics', 'AI', 'Ethical AI: Challenges and Opportunities'),
(2, '2024-02-15', 'Financial risk analysis techniques', 'Finance', 'Quantitative Risk Assessment'),
(3, '2024-03-20', 'Renewable energy advancements', 'Energy', 'The Future of Solar Power'),
(4, '2024-04-05', 'Medical innovations and healthcare', 'Health', 'AI in Disease Diagnosis'),
(5, '2024-05-12', 'Tech industry growth and trends', 'Tech', 'The Rise of Quantum Computing');

Table Institute:
INSERT INTO Institute (instituteName, address) VALUES
('MIT', 'Cambridge, MA, USA'),
('Stanford University', 'Stanford, CA, USA'),
('Harvard University', 'Cambridge, MA, USA'),
('Oxford University', 'Oxford, UK'),
('Tsinghua University', 'Beijing, China');

Table Author:
INSERT INTO Author (aid, name, instituteName) VALUES
(1, 'Alice Johnson', 'MIT'),
(2, 'Bob Smith', 'Stanford University'),
(3, 'Charlie Lee', 'Harvard University'),
(4, 'David Brown', 'Oxford University'),
(5, 'Eve Zhang', 'Tsinghua University');

Table Wrote:
INSERT INTO Wrote (aid, pid) VALUES
(1, 1),
(2, 2),
(3, 3),
(4, 4),
(5, 5);

Table Comment:
INSERT INTO Comment (poid, cid, email, content) VALUES
(101, 1, 'alice@example.com', 'Great insights on AI ethics!'),
(102, 2, 'bob@example.com', 'Interesting take on financial risk.'),
(103, 3, 'charlie@example.com', 'Renewable energy is the future!'),
(104, 4, 'david@example.com', 'I appreciate the healthcare analysis.'),
(105, 5, 'eve@example.com', 'Quantum computing is fascinating.');

Table LikedBy:
INSERT INTO LikedBy (poid, email) VALUES
(101, 'alice@example.com'),
(102, 'bob@example.com'),
(103, 'charlie@example.com'),
(104, 'david@example.com'),
(105, 'eve@example.com');

CPSC 304 Project Cover Page

University of British Columbia, Vancouver
Department of Computer Science

---

9. Acknowledgment of AI tools

Yes, we confirm that we use only one AI tool, ChatGPT, for grammar revision in our deliverables and for generating example data. Since manually creating a large volume of dummy data is challenging, we use AI-generated data as a starting point but carefully review and revise it to ensure consistency within our system (e.g., all references exist and align correctly).